# Reversium by 3Sci Labs

## 1 / 7 - Understand a package you received

Owner or project manager - Know what was delivered and what to ask next.

### 1. Choose your goal

Start > Understand the software

### 2. Select your package

ZIP > Analyze ZIP locally

### 3. Read and inspect

Review findings > Architecture > Evidence

### 4. Agree next steps

Coverage > Save & next steps

Take away: Readable HTML report plus questions for the developer.

Boundary: A feature explanation is supported code evidence, not proof that the feature works.

# Reversium by 3Sci Labs

## 2 / 7 - Move a project between AI coding tools

Builder changing tools or developers - Continue from an evidence-backed handover, for example Claude to Codex.

### 1. Save the source

Export your authorized code package from the original tool

↓

### 2. Map the delivery

Analyze > Architecture > Evidence

↓

### 3. Prepare the handover

Handover: setup clues, owners, gaps and next checks

↓

### 4. Continue elsewhere

Give the next developer the code, reports and selected context

Take away: Original code package kept separately, plus one reviewed handover ZIP with reports and optional annotations.

Boundary: Manual handover. Reversium does not migrate conversations, agent memory, prompts, secrets or accounts.

# Reversium by 3Sci Labs

## 3 / 7 - Take over maintenance

Incoming developer or agency - Identify first files, required services and unresolved setup tasks.

### 1. Analyze the delivery

Start > Prepare a developer handover

### 2. Follow one feature

Architecture > Workflow > Evidence

### 3. Review setup needs

Handover > Setup clues; Coverage > Setup & readiness

### 4. Assign and save

Set checklist owners; optionally include annotations in the handover pack

Take away: A scoped investigation and setup checklist for the exact delivery.

Boundary: Service references are clues. The developer still verifies versions, permissions, migrations and credentials.

# Reversium by 3Sci Labs

## 4 / 7 - Plan a change without guessing

Developer or maintainer - Find candidate components to inspect before editing.

### 1. Pick the starting file

Analyze > Change impact

### 2. Explore connections

Upstream and downstream candidate neighbors

### 3. Check each claim

Architecture map > Evidence > Coverage

### 4. Make and verify the change

Use your editor and normal tests; save a new baseline

Take away: Candidate impact notes and a source-backed test plan.

Boundary: Graph reachability is not proof that a component will be affected. Reversium does not edit or deploy code.

# Reversium by 3Sci Labs

## 5 / 7 - Check what changed in a new delivery

Owner, reviewer or developer - Compare a new package with a saved earlier analysis.

### 1. Keep a baseline

Save technical data (JSON) from the earlier analysis

### 2. Analyze the new ZIP

Create a separate current report

### 3. Compare reports

Select earlier JSON; inspect files, findings and map changes

### 4. Record the review

Check evidence; save comparison notes and next tests

Take away: Changed relationships and candidate effects, with disclosed omissions.

Boundary: Analyzer versions and packaging can change results. A difference is not automatically a defect.

# Reversium by 3Sci Labs

## 6 / 7 - Ask AI a focused question privately

Owner or developer needing an explanation - Share only selected evidence and check the answer.

### 1. Choose the evidence

AI: select provider, question and up to 20 findings

↓

### 2. Review the exact text

Prepare request; remove private details

↓

### 3. Choose how to submit

Online: manual handoff; local: eligible optional connection

↓

### 4. Check the answer

Match cited IDs/locations; record unsupported claims

Take away: A reviewed request and a human-checked explanation.

Boundary: Nothing is shared automatically. Online account-connected chat is disabled; provider terms/limits apply.

# Reversium by 3Sci Labs

## 7 / 7 - Verify behavior in a disposable local environment

Developer with an explicitly reviewed execution profile - Separate observed behavior from setup assumptions without touching live.

### 1. Review support first

Analyze package; review alerts and compatibility

### 2. Prepare isolation

Local prerequisites, trusted assets and fresh isolation checks

### 3. Run a bounded test

Fixed reviewed profile: real code, disposable data and accounts

### 4. Record and stop

Capture passes/blockers, stop guest and review matching evidence

Take away: Exact-package test record and separately recorded manual observations.

Boundary: Experimental local profile only, not any ZIP or a simulation. No production/live credentials. VM isolation cannot guarantee zero risk.